Template Method

Patrón de diseño de comportamiento

Caso de estudio: preparación de café y té en Java

Integrantes: _____

Materia / curso: _____

Fecha de presentación: _____

Trabajo académico: problema, diseño, implementación, comparación y práctica.

1. El problema

Supongamos que un programa prepara café y té. Los dos procesos tienen una secuencia general similar: hervir agua, preparar el ingrediente principal, servir y agregar un extra. Algunos pasos son iguales y otros cambian según la bebida.

Versión sin patrón: cada clase implementa todo el procedimiento por separado.

```
class PreparadorCafe {
    void preparar() {
        hervirAgua();
        filtrarCafe();
        servirEnTaza();
        agregarAzucar();
    }
}
class PreparadorTe {
    void preparar() {
        hervirAgua();
        colocarSaquito();
        servirEnTaza();
        agregarLimon();
    }
}
```

Dónde duele: hervir agua y servir en taza aparecen duplicados. Si cambia el procedimiento compartido, hay que encontrar y modificar cada implementación. Con más bebidas, aumenta la posibilidad de que las secuencias se desalineen: por ejemplo, una clase podría servir antes de preparar el ingrediente. También es más difícil garantizar un orden uniforme.

El problema no es que existan dos clases, sino que cada una controla por su cuenta una secuencia que debería mantenerse igual. Template Method permite centralizar el esqueleto del algoritmo y dejar variables solo los pasos necesarios.

Objetivo de diseño

Definir en una clase abstracta el orden general de preparación y delegar a las subclases los pasos que dependen de la bebida. En este ejemplo, el orden será: hervir agua → preparar bebida → servir en taza → agregar extra.

2. Por qué Template Method y no otro

La solución simple deja de ser conveniente cuando hay varias variantes que comparten una secuencia estable. Copiar y pegar pasos comunes aumenta el coste de mantenimiento y permite que las implementaciones diverjan. Template Method mueve los pasos comunes a la superclase y define un único método plantilla que los llama en orden.

Comparación explícita con Strategy

Ambos patrones separan lo que permanece estable de lo que varía, pero organizan la variación de manera distinta.

Criterio	Template Method	Strategy
Mecanismo	Herencia: subclases redefinen pasos	Composición: un objeto delega en una estrategia
Qué varía	Pasos concretos de un algoritmo común	Algoritmos intercambiables, a menudo completos
Quién controla el flujo	La clase abstracta	El contexto llama a la estrategia
Cambio en ejecución	La variante normalmente queda ligada al tipo concreto	Se puede reemplazar la estrategia durante la ejecución
Coste	Jerarquía de herencia; la estructura puede ser rígida	Más objetos/interfaces y selección de estrategia
Conviene cuando	El orden debe mantenerse fijo y cambian algunos pasos	Se necesita elegir o intercambiar algoritmos

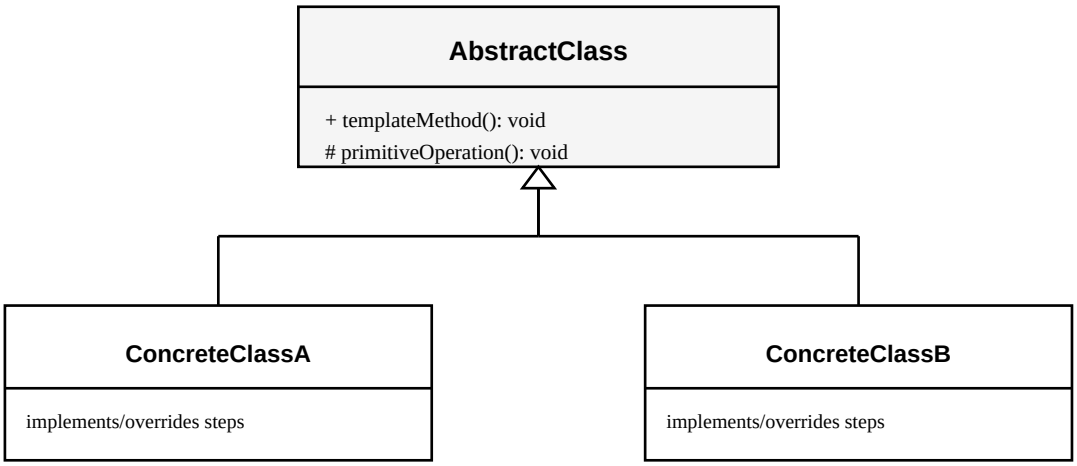
En el caso de las bebidas, el orden de los pasos es parte de la regla del proceso. No queremos que una subclase decida servir antes de hervir el agua; queremos permitirle definir cómo se prepara la bebida. Por eso Template Method encaja mejor que Strategy para este ejemplo didáctico.

Cuándo elegir Strategy en cambio: si el programa necesitara intercambiar algoritmos completos en tiempo de ejecución, o si el comportamiento variable tuviera que combinarse con distintos contextos sin heredar de una clase

base, Strategy ofrecería mayor flexibilidad. Template Method no es universalmente superior; se elige por la estabilidad del esqueleto.

3. Estructura genérica del patrón

Template Method es un patrón de comportamiento. La clase abstracta declara el método plantilla y los pasos del algoritmo; algunos pasos se implementan en la clase base y otros se dejan abstractos o como hooks opcionales.



La clase abstracta fija el flujo; las subclases completan los pasos variables.

AbstractClass: representa la clase base. Contiene el método plantilla y los pasos comunes o variables.

templateMethod(): define el esqueleto del algoritmo y llama a los pasos en un orden determinado. En Java suele declararse final si no debe permitirse que las subclases alteren la secuencia.

primitiveOperation(): paso abstracto que las subclases deben implementar.

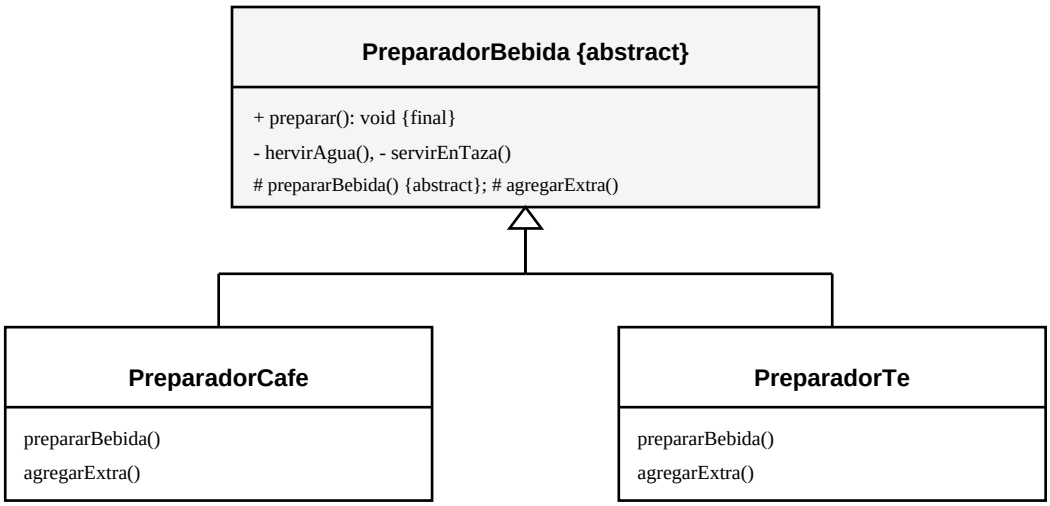
hook(): paso opcional con implementación por defecto; la subclase puede redefinirlo si lo necesita.

ConcreteClass: clase concreta que implementa los pasos variables sin reemplazar el método plantilla.

Notación UML: la flecha de generalización tiene línea continua y triángulo hueco apuntando a la superclase.

4. Ejemplo concreto en Java

La clase PreparadorBebida representa la clase abstracta. El método preparar() es el Template Method. Los métodos hervirAgua() y servirEnTaza() contienen pasos comunes. prepararBebida() es abstracto y obligatorio; agregarExtra() es un hook opcional.



Cafe: filtrar café / azúcar. Té: saquito / limón.

Código clave: clase abstracta

```
public abstract class PreparadorBebida {
    public final void preparar() {
        hervirAgua();
        prepararBebida();
        servirEnTaza();
        agregarExtra();
    }

    private void hervirAgua() {
        System.out.println("Hirviendo agua");
    }

    protected abstract void prepararBebida();

    private void servirEnTaza() {
        System.out.println("Sirviendo en una taza");
    }

    protected void agregarExtra() {
        // Paso opcional por defecto.
    }
}
```

Explicación: abstract impide crear directamente una bebida genérica. public final void preparar() es la secuencia estable y no puede sobrescribirse en las subclases. Los métodos privados son detalles comunes que no necesitan variar. prepararBebida() es abstracto, así que cada subclase debe implementarlo. agregarExtra() tiene una implementación vacía para que el paso sea opcional.

Código clave: subclases

```
public class PreparadorCafe extends PreparadorBebida {
    @Override
    protected void prepararBebida() {
        System.out.println("Filtrando cafe");
    }

    @Override
    protected void agregarExtra() {
        System.out.println("Agregando azucar");
    }
}

public class PreparadorTe extends PreparadorBebida {
    @Override
    protected void prepararBebida() {
        System.out.println("Colocando saquito de te");
```

```
    }

    @Override
    protected void agregarExtra() {
        System.out.println("Agregando limon");
    }
}
```

En UML, PreparadorCafe y PreparadorTe heredan de PreparadorBebida. Ambas redefinen los pasos variables. El resto del código está en el repositorio entregado junto con este documento.

5. Uso real y cuándo NO usarlo

Un caso realista es un proceso de importación de archivos: validar entrada, abrir el archivo, leer datos, transformar registros, cerrar recursos y reportar el resultado. El esqueleto puede ser común mientras la lectura de CSV, JSON o XML cambia. También se usa en clases base de frameworks que definen un ciclo de vida y permiten sobrescribir determinados pasos.

Cuándo no usarlo: si una aplicación solo prepara café y no habrá otras bebidas, introducir una clase abstracta y métodos protegidos agrega estructura sin resolver una duplicación relevante. Tampoco conviene si cada bebida necesita un orden completamente distinto, o si el algoritmo cambia en muchos ejes independientes. En esos casos, una función sencilla, composición o Strategy pueden ser más claros.

Ventajas

• Reduce la duplicación de pasos comunes.
• Centraliza y protege el orden del algoritmo.
• Facilita agregar una nueva variante creando otra subclase.
• Permite que las subclases se concentren en las diferencias.

Costes y límites

• Acopla las subclases a una jerarquía de herencia.
• El esqueleto puede ser demasiado rígido para procesos muy diferentes.
• Una modificación de los métodos protegidos puede afectar a todas las subclases.
• Una jerarquía excesiva dificulta entender el diseño.

6. Fuentes

1. Refactoring.Guru. “Template Method”. https://refactoring.guru/design-patterns/template-method

2. Refactoring.Guru. “Template Method en Java”. https://refactoring.guru/design-patterns/template-method/java/example

3. Refactoring.Guru. “Strategy”. https://refactoring.guru/es/design-patterns/strategy

4. Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

5. Apuntes de la materia: consigna de trabajo práctico sobre patrones de diseño (material proporcionado por la cátedra).

Nota: el ejemplo de café y té de este trabajo es didáctico y está adaptado para mostrar el patrón; las fuentes se usan para fundamentar la intención y estructura del patrón.